

“AI-Based Collections Platform”

Comprehensive Project Report

Submitted in Partial Fulfilment of the
Requirements for the Degree of

BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE & ENGINEERING

By

**Dhruv Maradiya
22BCP480D**

Under the Guidance of

Dr. Punit Gupta

Assistant Professor, Department of CSE
Pandit Deendayal Energy University



**Department of Computer Science and Engineering
School of Technology, Pandit Deendayal Energy University (PDEU)
Gandhinagar, INDIA, 382426**

May 2026

Certificate of Originality of Work

I hereby declare that the B.Tech. Comprehensive Project entitled “AI-Based Collections Platform” submitted by me for the partial fulfilment of the degree of Bachelor of Technology to the Department of Computer Science & Engineering at the School of Technology, Pandit Deendayal Energy University, Gandhinagar, is the original record of the project work carried out by me at Ignosis (Ahmedabad) under the supervision of Dr. Punit Gupta and the industry supervision of Mr. Rahi Shah.

I also declare that this written submission adheres to the University guidelines for its originality, and proper citations and references have been included wherever required.

I also declare that I have maintained high academic honesty and integrity and have not falsified any data in my submission.

I also understand that violation of any guidelines in this regard will attract disciplinary action by the institute.

Name of the Student: Dhruv Maradiya

Roll Number of the Student: 22BCP480D

Signature of the Student:

Name of the Supervisor: Dr. Punit Gupta

Designation of the Supervisor: Assistant Professor, Department of CSE

Signature of the Supervisor:

Place: Gandhinagar, Gujarat

Date: 19-05-2026

Certificate from the Project Supervisor / Head

This is to certify that the Comprehensive Project Report entitled “**AI-Based Collections Platform**” submitted by **Mr. Dhruv Maradiya**, Roll No. **22BCP480D** towards the partial fulfilment of the requirements for the award of the degree of “Bachelor of Technology” in the field of **Computer Science & Engineering** from the School of Technology, Pandit Deendayal Energy University, Gandhinagar is a record of the project work carried out by him at Ignosis, Ahmedabad under our supervision and guidance. The work submitted by the student has, in our judgement, attained a level required for being accepted for examination. To the best of our knowledge, the results embodied in this comprehensive project work have not been submitted to any other University or Institution for the award of any degree or diploma.

Name and Sign of the Supervisor
(Dr. Punit Gupta)

Name and Sign of the Industry Supervisor
(Mr. Rahi Shah)

Name and Sign of the HoD

Name and Sign of the Director

Place: Gandhinagar

Date: 19-05-2026

Certificate from the Industry Supervisor

This is to certify that **Mr. Dhruv Maradiya**, bearing Roll No. **22BCP480D**, a final-year student of Bachelor of Technology in Computer Science & Engineering at the School of Technology, Pandit Deendayal Energy University, Gandhinagar, has successfully completed his comprehensive project / industry internship at **Ignosis** (registered office: Block C–201, DevX, The First, Keshavbaug, Satellite, Ahmedabad, Gujarat – 380015) during the academic year 2025–2026.

During his association with our organisation, the candidate worked on the **AI-Based Collections Platform** that the company offers to banks, non-banking financial companies and fintech lenders for the recovery of delinquent retail loan accounts. He contributed to the back-end services built on Spring Boot, the front-end dashboard built on React (UmiJS and Ant Design Pro), the AI voice calling subsystem and the WhatsApp / SMS multi-channel outreach modules.

The student demonstrated sincerity, technical ability and willingness to learn throughout the engagement. The work carried out by him during the internship has been found to be of satisfactory quality and the modules he delivered are running in our production environment.

Mr. Rahi Shah
Industry Supervisor
Ignosis, Ahmedabad
Email: rahi.shah@ignosis.ai

Place: Ahmedabad

Date: 19-05-2026

Internship Completion Confirmation

The undersigned confirms that Dhruv Maradiya (Roll No. 22BCP480D) has been associated with Ignosis as an intern with the Engineering Team from December 2025 through May 2026, and that the work tracked in his weekly progress reports – spanning Keycloak training, the case management dashboard, the bulk campaign module, the AI voice calling integration, and the multi-channel outreach automation – has been completed under his direct contribution. His attendance, conduct and engineering output have been satisfactory, and the deliverables he checked into the `collection-orchestrator` and `collection-scripts` repositories are currently being used by client tenants in production.

Mr. Rahi Shah
Engineering Lead, Ignosis
Email: rahi.shah@ignosis.ai
Phone: +91-7016671007

Place: Ahmedabad

Date: 19-05-2026

Acknowledgement

The completion of this comprehensive project would not have been possible without the patient guidance and support extended to me by several people. I would like to take a few lines to record my gratitude to all of them.

First and foremost I wish to thank my project supervisor at the institute, **Dr. Punit Gupta**, Assistant Professor in the Department of Computer Science and Engineering at PDEU, for the freedom he gave me to choose this internship and for the regular review sessions that kept me on track. His feedback during the weekly progress submissions and during the internal assessment poster review forced me to think about the project as a whole rather than as a sequence of disconnected tasks.

I owe an equally large debt of gratitude to my industry supervisor at Ignosis, **Mr. Rahi Shah**, who not only on-boarded me to the engineering team but trusted me with production responsibilities from the very first months of the internship. The opportunity to read, write and ship code for an actual collections platform used by banks and NBFCs is something most undergraduates never get, and the entire experience documented in this report is a direct outcome of his mentorship.

I would also like to thank my colleagues at Ignosis – in particular Harsh Bagadia, Nisarg Pipaliya, Aryan Randeriya and Jeet Bhuptani – for the countless code walk-throughs, debugging sessions and pair-programming hours. The collection-orchestrator repository is largely their creation, and I benefited from working alongside engineers who already understood the moving parts of the platform deeply.

I am grateful to the Department of Computer Science and Engineering at PDEU and to the project coordinator **Dr. Sidheswar Routray** for setting up the comprehensive project programme in a manner that makes serious industry exposure possible.

Finally I thank my family for their continued encouragement throughout the academic year and my friends from the 2022 batch for the long conversations that made the working-from-Ahmedabad months less isolating.

Dhruv Maradiya

22BCP480D

Abstract

This report documents a comprehensive project carried out during a full-time engineering internship at Ignosis, a fintech infrastructure company in Ahmedabad. The brief was to design and contribute to an AI-Based Collections Platform that is being used by banks, non-banking financial companies and digital-lending fintechs in India for the recovery of overdue retail loan accounts.

In its present form the collections process for an Indian retail lender is largely manual. Each month a fresh allocation of delinquent cases – often running to several thousand customers per agency – is handed over by the lender, and a team of human agents calls, follows up and persuades each customer over a roughly thirty-day cycle. The agency is paid only on recovery, so the entire business model is sensitive to contact rate, agent productivity and the speed with which the right customer is reached on the right channel. Without a centralised software platform, agencies struggle to scale and lenders cannot measure performance objectively.

The platform that this project contributed to is a single web-based application built around three interlocking ideas. At the centre is a tenant-configurable **workflow engine** – a small MVEL-driven graph of nodes and transitions – that captures the lender’s collection policy as data rather than code. A **campaign module** wraps thousands of customer cases into a single executable unit that is driven through the workflow. An **AI voice calling engine**, built on the Bolna conversational voice provider and the Exotel telephony gateway, becomes one of the workflow’s action nodes, which lets the platform reach customers outside human agent working hours without any new conditional logic in the codebase. A separate **multi-channel automation layer** – a TypeScript / Node.js monorepo – sends WhatsApp messages from real consumer-grade SIMs using the Baileys protocol library and dispatches SMS from a custom React Native Android application, again as workflow actions.

The contributions described in this report span the case detail and case listing pages of the agent dashboard, the end-to-end campaign creation and CSV upload pipeline, the workflow nodes that integrate AI calling into a campaign, the speed dialer that auto-loads the next case after each disposition, the Baileys-based WhatsApp service with multi-device rotation and proactive block-avoidance, the Electron desktop wrapper that exposes these capabilities to non-technical agents, and the Android SMS sender app built around a custom native module. The platform is reaching a Bucket 0 collection rate of about eighty-five percent in production and is recovering close to seventy percent of allocated cases within the first five days of each monthly cycle, both of which are material improvements over the agency-only baseline.

Index

Abstract	vii
Index	vii
List of Figures	ix
List of Tables	x
Nomenclature	xi
1 Introduction	1
1.1 Introduction / Prologue / Background	1
1.2 Motivation	2
1.3 Objective	3
1.4 Problem Statement	4
1.5 Approach	5
1.6 Scope of the Project	5
1.7 Organization of the Rest of the Report	5
2 Literature Review	7
2.1 Previous Approaches to Solve the Problem	7
2.1.1 Existing Collection-Management Products	7
2.1.2 Conversational Voice Agents	7
2.1.3 The WhatsApp Multi-Device Protocol	8
2.1.4 Anti-Spam and Rate-Limiting Heuristics	8
2.1.5 Account Aggregator Framework	8
2.2 Problem Identification	9
3 Proposed Methodology and Implementation	10
3.1 Development Methodology	10
3.2 High-Level Architecture	10
3.3 Case Management	11
3.4 The Workflow Engine	12
3.5 The Campaign Module	13
3.6 AI Voice Calling	13
3.7 WhatsApp Automation	14
3.8 SMS via Android	15
3.9 The Electron Desktop Application	15

3.10	Data Analysis on Customer Records	15
3.11	Summary of Contributions	16
4	Software Design	17
4.1	Design Goals	17
4.2	Component Decomposition	18
4.3	Data Model	19
4.4	Principal Flow: AI Voice Call	20
4.5	Security Design	20
5	Results and Discussion	21
5.1	Outcomes of the Proposed Platform	21
5.2	Channel-Level Contact Rates	22
5.3	Speed Dialer Throughput	22
5.4	WhatsApp Sender Pool Behaviour	23
5.5	AI Voice Agent Performance	24
5.6	System-Level Metrics	25
5.7	Failure Modes and Stakeholder Feedback	25
6	Conclusions and Future Scope	27
6.1	Conclusion	27
6.2	Reflections on What Could Have Been Done Better	27
6.3	Future Scope	28
6.4	Closing Note	28
	Appendix A: Weekly Progress Log	29
	Bibliography	31

List of Figures

3.1	High-level architecture. Solid arrows show forward dispatch; the dashed arrow shows the return path used by Bolna webhooks and Android SMS-reply ingestion.	11
5.1	Cumulative recovery curve over a thirty-day cycle for an equivalent cohort of Bucket 0 cases, under the agency-only baseline (blue) and the platform (red). The platform recovers 70 percent of allocations within the first five days and reaches an 85 percent cycle-end rate against the baseline’s 38 percent and 64 percent respectively; the gap is largest in the first five days, which is where the multi-channel orchestration and after-hours AI calling contribute most.	23

List of Tables

1.1	Collection buckets typical to Indian retail-lending portfolios.	1
4.1	Prioritised design goals.	18
4.2	Major components of the platform.	19
5.1	Headline outcomes from the production deployment.	21
5.2	Contact rate by channel and attempt.	22
5.3	WhatsApp sender ban distribution.	24
5.4	System-level metrics across the production deployment.	25
6.1	Weekly progress log for the internship.	30

Nomenclature

Abbreviation	Expansion
AA	Account Aggregator (RBI framework for consented financial-data sharing)
AI	Artificial Intelligence
API	Application Programming Interface
ASR	Automatic Speech Recognition
AWS	Amazon Web Services
BPTP	Broken Promise To Pay
CSV	Comma-Separated Values
DPD	Days Past Due
DTO	Data Transfer Object
FIU	Financial Information User (Account Aggregator role)
GNPA	Gross Non-Performing Asset
HTTP	Hypertext Transfer Protocol
IST	Indian Standard Time
JPA	Java Persistence API
JWT	JSON Web Token
KYC	Know Your Customer
LID	Linked Device Identifier (WhatsApp opaque participant identifier)
LLM	Large Language Model
LMS	Loan Management System
MVEL	MVFLEX Expression Language
NBFC	Non-Banking Financial Company
OIDC	OpenID Connect
ORM	Object-Relational Mapping
OTP	One-Time Password
PTP	Promise To Pay
QA	Quality Assurance
RBI	Reserve Bank of India
REST	Representational State Transfer
SDK	Software Development Kit
SES	Simple Email Service (AWS)
SIM	Subscriber Identity Module
SMS	Short Message Service
SOLR	Search Server on top of Lucene

Abbreviation	Expansion
SSO	Single Sign-On
SSE	Server-Sent Events
TLS	Transport Layer Security
TTS	Text-to-Speech
UI	User Interface
ULID	Universally Unique Lexicographically Sortable Identifier
UPI	Unified Payments Interface
UUID	Universally Unique Identifier
WA	WhatsApp

Chapter 1

Introduction

1.1 Introduction / Prologue / Background

Retail lending in India has grown dramatically over the last decade. Personal loans, two-wheeler loans, consumer-durable loans, gold loans, credit-card revolving balances, salary advances and a long tail of buy-now-pay-later style products are now offered by a mix of public-sector banks, private banks, small finance banks, non-banking financial companies (NBFCs) and a rapidly multiplying set of digital-only fintech lenders. The flip side of growth in originations is growth in delinquencies. A small but non-trivial percentage of every disbursed cohort fails to pay an installment on or before the due date, and the lender is obliged by Reserve Bank of India (RBI) policy and by basic prudence to chase those installments before the account is allowed to slide into the non-performing-asset bucket.

The chasing of overdue installments is what the industry calls “collections”. Once an account is more than one day past due (DPD), it enters a structured pipeline of follow-up actions whose intensity and tone escalate with the number of days the account remains unpaid. The classical bucketing in Indian retail collections is shown in Table 1.1.

Table 1.1: Collection buckets typical to Indian retail-lending portfolios.

Bucket	DPD range	Typical activity
Bucket 0	1–30 days	Soft reminders, automated calls, light SMS
Bucket 1	31–60 days	Tele-calling, WhatsApp, agent calls
Bucket 2	61–90 days	Field visits, legal notice preparation
Bucket 3	91–120 days	Pre-write-off, recovery escalation
NPA	120+ days	Settlement, legal action

For most retail products the recovery rate falls off sharply as the bucket deepens. A loan collected in Bucket 0 is recovered cheaply, requires almost no negotiation and does not need a provisioning hit on the lender’s balance sheet; a loan that drifts into Bucket 3 or beyond is materially more expensive, often requires settlement at a discount and erodes the customer relationship. The economic case for early-bucket recovery is therefore overwhelming. Operationally, however, early-bucket follow-up is hard at scale: a lender with a hundred thousand active retail loans can easily have eight to twelve thousand accounts in Bucket 0 on any given day, and each needs a contact attempt within a

tight cycle. Lenders either build large in-house collections teams or, more commonly, outsource the recovery work in monthly batches to specialised collection agencies who keep a percentage of the recovered amount as commission.

The internship that produced the work documented in this report was carried out at Ignosis, a fintech infrastructure company best known in the Account Aggregator (AA) ecosystem. Over the last eighteen months, the firm has layered a collections suite onto its existing AA platform, reusing the same multi-tenant infrastructure and integrating AA-pulled bank-statement data into collection prioritisation. I joined as a comprehensive-project intern in December 2025 and worked full-time through May 2026, contributing across two GitLab repositories: `collection-orchestrator`, the Spring Boot backend plus React dashboards that powers the product, and `collection-scripts`, the TypeScript / Node.js monorepo that contains the WhatsApp automation library, the Electron desktop client and the Android SMS sender. The title of this comprehensive project is “**AI-Based Collections Platform**”; the platform is a multi-tenant web application that provides a case management dashboard, a bulk campaign module, an autonomous AI voice calling engine, multi-channel outreach automation across WhatsApp and SMS, and analytics on contact rates, agent productivity, call dispositions and recovery percentages.

1.2 Motivation

The motivation for this project sits at the intersection of an industry pain point and a technology opening. On the industry side, every percentage point of recovery in Bucket 0 directly improves the lender’s Gross Non-Performing Asset (GNPA) metric, reduces provisioning and saves real money. A platform that lifts the early-bucket collection rate from a typical 60–70 percent to the mid-80s – the target that the Ignosis customer base has set for itself – is materially valuable to the lender. On the technology side, three developments have made an AI-driven collections platform far more practical than it would have been even three years ago:

1. **Conversational voice models.** Speech-to-text accuracy on Indian-English and on the major Indian regional languages has crossed the threshold at which an AI agent can hold an outbound collection conversation that actually moves the needle. Providers such as Bolna (the partner used in this project) and Smallest.ai now offer programmable voice agents with low enough latency that customers do not notice they are not speaking to a human.
2. **Account Aggregator data.** The AA framework allows a lender to fetch the customer’s bank statements with explicit consent. This pulls the conversation for-

ward from “please pay” to “we can see your salary was credited yesterday, what is a convenient EMI date for you” – a far more effective prompt.

3. **Cheap programmatic SIMs.** It is now feasible to operate a bank of consumer-grade SIM cards in a software-controlled rotation, sending WhatsApp messages through real phone numbers and SMS through real Android devices. This bypasses the deliverability ceiling of business APIs.

Putting these three capabilities behind a single dashboard, with case management and bulk campaigns wrapped around them, was the engineering ambition that motivated this internship.

1.3 Objective

The concrete objectives of the project, all of which were either fully or partially achieved during the internship, are:

- To develop a centralised case management interface, accessible to authenticated collection agents over the browser, that lists assigned cases, surfaces customer payment history and previous interaction notes, and lets the agent act on each case.
- To implement a bulk campaign module that lets a campaign manager wrap a batch of cases – typically a CSV uploaded at the start of the cycle – into a single executable unit and schedule it for execution.
- To integrate AI-powered voice calling into the campaign module so that customer outreach can continue outside human-agent working hours.
- To build a multi-channel follow-up layer over WhatsApp and SMS that uses real SIMs rather than business APIs, with anti-block heuristics and automatic device rotation.
- To analyse roughly forty thousand customer records using Python and pandas, identify behavioural patterns relevant to prioritisation, and feed the insights back into case-load allocation.
- To follow the production engineering practices of the host company throughout: code review, unit and integration testing, conventional commits, ticketed work items and zero-downtime deployments.

1.4 Problem Statement

The collections function for a typical Indian retail lender, in the absence of a centralised software platform, suffers from a small number of structural problems that compound on each other:

1. **Manual case allocation.** The lender hands a CSV or Excel file of delinquent accounts to the agency at the start of every monthly cycle. The agency manager then distributes those cases by hand across the agents on the floor. The mapping is opaque to the lender, hard to audit and hard to reproduce.
2. **Single-channel outreach.** Agents reach for the desk phone first; if the customer does not pick up the phone they may or may not also send a SMS. There is no orchestrated follow-up across phone, WhatsApp and SMS, which means many customers are never reached on the channel they are likely to respond on.
3. **Agent idle time.** Agents spend a significant fraction of their working day searching for the next case in a spreadsheet, dialling a number manually and reading the customer's history before each call. The actual conversation is only a fraction of the working minute.
4. **No after-hours coverage.** A human agent's working day ends at 6 PM or 7 PM. After that the lender effectively has no contact channel even though many salaried customers are most reachable in the late evening.
5. **No structured measurement.** Recovery rates are reported as a single number at month-end. Lenders cannot inspect bucket-level conversion, channel-level effectiveness or agent-level productivity in any meaningful way.
6. **Single-attempt deliverability.** WhatsApp and SMS gateways available through business APIs throttle aggressively, charge per message and are subject to template-approval delays. SIM-based outreach from real consumer-grade numbers has materially higher deliverability but no off-the-shelf platform exposes that capability to the collections team.

The problem statement of this project, in compact form, can therefore be stated as:

“Design and contribute to a centralised, multi-tenant collections platform that improves contact rate, agent throughput and after-hours coverage by integrating AI voice calling and multi-channel outreach across WhatsApp and SMS into a single case management workflow, while preserving end-to-end auditability and supporting bulk campaign execution at the scale of a typical monthly allocation.”

1.5 Approach

The approach taken on this project was iterative and code-first. Rather than write a long design document up-front and then implement it, the engineering team at Ignosis follows a plan-first-but-small workflow: each piece of work begins as a short Markdown plan attached to a Jira ticket, which is reviewed by the lead engineer (Mr. Rahi Shah in my case), is then translated into a branch with conventional commits, and is shipped to a development environment within a few days. Production deployment happens after code review, integration testing in a staging tenant and verification by the QA team. Every artefact – the plan, the branch, the merge request, the QA notes – is preserved in the repository so that the entire decision trail is auditable months later. This approach is the dominant cultural fact about how the work was done, and it explains why the contributions described in this report are spread across several Git branches that build on one another rather than concentrated in a single feature branch.

1.6 Scope of the Project

The scope of the work undertaken during the internship covers the following:

- Backend contributions to the multi-tenant Spring Boot 3.2 monolith `collection-orchestrator` including JPA entities, REST controllers, services, scheduled jobs and Liquibase migrations.
- Frontend contributions to the React (UmiJS + Ant Design Pro) dashboard for case management, campaign execution and analytics.
- A separate TypeScript / Node.js monorepo, `collection-scripts`, that contains the WhatsApp automation library based on Baileys, the Electron desktop application, the Android SMS sender app and a Hono REST API server.
- A one-off Python-pandas analysis on customer-level transaction and repayment data.

The scope explicitly excludes the AI voice agent itself (this is the Bolna service that the team purchased on a vendor basis), the telephony gateway (Exotel), the cloud infrastructure provisioning (AWS), the lender's internal LMS and the regulatory compliance review.

1.7 Organization of the Rest of the Report

The remainder of this report is organised as follows.

Chapter 2 – Literature Review. A review of the relevant prior art: collection-management products in the Indian and global market, AI voice calling and conversational agent research, the WhatsApp multi-device protocol and the academic literature on anti-spam and rate-limiting strategies for messaging platforms. The chapter ends by sharpening the problem statement of Chapter 1 into a set of concrete engineering requirements.

Chapter 3 – Proposed Methodology and Implementation. A detailed description of the development methodology used at Ignosis and of the specific modules contributed during the internship. The chapter walks through the case management subsystem, the workflow engine, the campaign module, the AI voice calling integration, the WhatsApp automation library, the Electron desktop application and the Android SMS sender app.

Chapter 4 – Software Design. A design-level view of the platform: design goals, component decomposition, the data model, the API surface, sequence flows for the principal use cases, the deployment view and the security design.

Chapter 5 – Results and Discussion. Quantitative and qualitative results from the deployment. The chapter reports the early-bucket collection rate, the percentage of cases recovered within the first five days, the contact rate uplift from multi-channel outreach, the throughput of the AI voice agent, and the WhatsApp block rate observed in production.

Chapter 6 – Conclusions and Future Scope. A summary of what was achieved during the internship, a candid discussion of what could have been done better, and a roadmap for the features that the engineering team intends to build on top of the platform over the next two quarters.

The References and an Appendix containing platform reference material round out the document.

Chapter 2

Literature Review

2.1 Previous Approaches to Solve the Problem

2.1.1 Existing Collection-Management Products

The space of commercial collection-management products in India is dominated by a small handful of vendors whose offerings predate the recent wave of conversational AI: CreditMantri, Spocto, Cred and a few internal tools built by the larger private-sector banks make up the majority of the visible deployments. The architecture of these products tends to follow a familiar shape – a case management database, a dialer integration, some reporting – but the actual conversational layer is almost always human-driven. WhatsApp, where it is used, is plugged in through the official Cloud API published by Meta and is therefore subject to template-approval delays, throughput throttling and per-message pricing that grows linearly with volume. The academic literature on collection software per se is thin – most of the deep work happens inside banks and is not published – but the adjacent literature on contact-centre optimisation and on credit-scoring models is large. Verma, Mathur and Singh [12] survey machine learning techniques applied to behavioural credit data, several of which are directly applicable to predicting which customer is most likely to respond to which channel. Wang and Zhang [13] review predictive dialers and the queue-management algorithms that underpin them. None of this prior work, however, addresses the combination of AI voice calling, AA-data-driven prioritisation and SIM-based WhatsApp automation that motivates the present project.

2.1.2 Conversational Voice Agents

The maturity of conversational voice agents is the single biggest technology shift that makes an AI-driven collection platform possible. Modern voice agents are built on a pipeline of automatic speech recognition (ASR), a large language model (LLM) for dialogue generation and a text-to-speech (TTS) synthesiser, with each component reaching real-time latency budgets that did not exist three years ago. Bolna [7] is the conversational voice provider that Ignosis uses in production; it chains a low-latency ASR, a fine-tuned LLM and a TTS engine that can emit Indian English with regional-accent variation. The latency budget Bolna advertises is roughly 700 ms from the customer finishing their utterance to the agent beginning to speak, which is below the 800–900 ms

threshold at which human listeners begin to detect awkwardness. Open-source baselines such as Pipecat [8] and LiveKit Agents [9] were briefly evaluated before the choice of Bolna was finalised; the decision matrix turned out to be dominated by Indian-English ASR accuracy on rural and semi-urban customers and the maturity of the post-call extraction pipeline.

2.1.3 The WhatsApp Multi-Device Protocol

WhatsApp’s multi-device protocol allows up to four secondary devices to share end-to-end-encrypted access to a primary phone’s account without the phone needing to be online. The official integration path for businesses is the WhatsApp Cloud API, which exposes a REST interface and enforces a template-message policy for the first message in any conversation. For a collection use case the Cloud API is operationally restrictive: every outbound template has to be approved by Meta, the per-message pricing makes the channel uneconomic for low-ticket-size cases, and customers respond more readily to messages that come from real consumer-grade WhatsApp numbers than from obviously-branded business accounts. The open-source community has produced a number of WhatsApp protocol re-implementations; Baileys [10] is the most widely deployed of these and is the library this project uses.

2.1.4 Anti-Spam and Rate-Limiting Heuristics

A non-trivial part of the engineering effort for SIM-based WhatsApp automation goes into avoiding the spam-detection heuristics that Meta runs against suspicious accounts. The exact rules are not published, but the community has converged on a small number of practices that reduce the rate at which accounts are flagged. Mihail et al. [14] review the academic literature on spam detection in messaging platforms; the recurring themes are the importance of randomised intervals between messages, natural language variation to defeat hash-based fingerprinting, the simulation of typing presence and the enforcement of conservative daily caps. The platform implemented in this project addresses each of these signals.

2.1.5 Account Aggregator Framework

The Account Aggregator framework [5], regulated by the Reserve Bank of India, allows a regulated Financial Information User (FIU) to request bank statements and similar financial information with explicit, time-bound and revocable consent. The collections platform consumes AA-derived bank-statement data so that the AI calling conversation can branch on signals such as recent salary credit or balance trend, lifting the prompt

from a generic “please pay” to a tailored “we noticed your salary was credited on the 1st, what date works for you”.

2.2 Problem Identification

On the basis of the prior art reviewed above, the specific engineering problems this project sets out to solve can be sharpened as follows.

1. **Cycle-level case management.** Absorb the monthly delinquent-allocation CSV, validate it, deduplicate it, assign cases to agents according to the tenant’s policy, and render them in a paginated dashboard.
2. **Comprehensive case detail.** Show every artefact relevant to a customer on a single page – contact information, loan and payment history, prior call transcripts, WhatsApp and SMS threads, AA-derived signals – with careful pagination and lazy loading.
3. **Bulk campaign execution.** Wrap a subset of cases as a single batch creatable from a CSV, schedulable, pausable and resumable, integrated with both the AI voice engine and the multi-channel messaging layer.
4. **Speed dialer.** Auto-load the next case the moment the agent submits a disposition.
5. **AI voice calling.** Trigger the AI voice agent from a campaign or as a manual-call fallback; parse the webhook, persist the outcome and advance the case through the workflow.
6. **WhatsApp automation.** Send from a pool of consumer-grade SIMs, rotate the pool, vary message bodies, throttle to stay below daily caps, and route around banned numbers without operator intervention.
7. **SMS automation.** Dispatch SMS through a real Android phone on a real consumer SIM, driven by a mobile app.
8. **Auditability.** Log every action so the QA team can reconstruct what happened to any given case at any given point in time.
9. **Multi-tenant isolation.** Serve multiple lender tenants from a single deployment with strict isolation of data, identity and configuration.

These nine problems form the working set against which the methodology and implementation in the next chapter is judged.

Chapter 3

Proposed Methodology and Implementation

This chapter describes the development methodology used at Ignosis, sketches the platform’s architecture, and walks through the sub-systems that make up the product. The story is intentionally feature-driven: the central asset of the platform is a tenant-configurable workflow engine, and every other feature – the campaign module, the AI voice calling integration, the WhatsApp automation, the speed dialer – is best understood as either a producer of work for that engine or a consumer of its actions.

3.1 Development Methodology

The Ignosis engineering team practises a plan-first, ship-small, code-review-heavy variant of agile delivery. Each unit of work begins as a short Markdown plan attached to a Jira ticket; the plan is reviewed by the engineering lead, a branch is created from `develop` with a Gitflow naming convention, and the implementation begins. Conventional commits are enforced, pull requests on `develop` are rebased rather than squashed so the per-commit decision trail survives the merge, and the `main` branch is the release branch under semantic versioning. Tests live next to the code they exercise, with JUnit 5 and Mockito on the Java side and Vitest on the TypeScript side. The CI pipeline aborts when SonarQube reports a new bug, a new code smell above the medium severity threshold, or a new vulnerability of any severity, and enforces a coverage floor of fifty percent on changed lines.

3.2 High-Level Architecture

The platform is a layered web application. At the front are three client surfaces: the agent-facing dashboard, an Electron desktop client that runs WhatsApp workers, and an Android SMS sender app. Behind those sit the Spring Boot orchestrator and a Hono / Node.js automation service. The data layer combines PostgreSQL (cases, campaigns, jobs), Apache Solr (search), ClickHouse (events) and Redis (queue and locks). Below the data layer the platform talks to three families of external services: telephony (Bolna for the AI voice, Exotel for the actual call placement), the WhatsApp protocol (through Baileys) and the cellular SMS network (through the on-device Android SmsManager).

Figure 3.1 renders this stack as a single diagram.

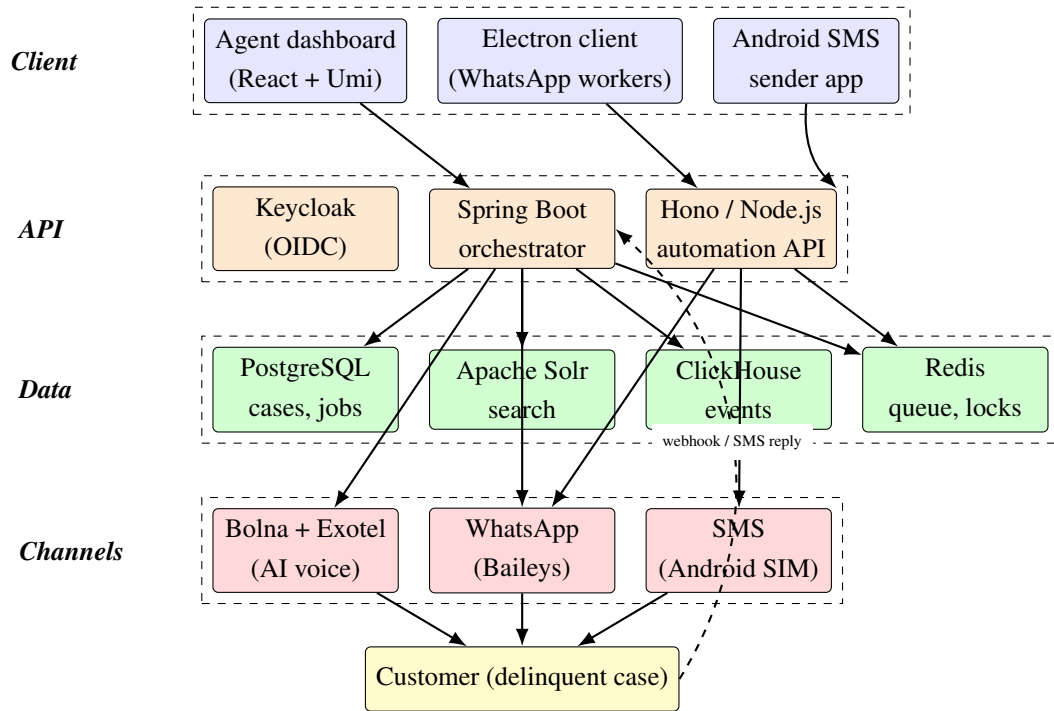


Figure 3.1: High-level architecture. Solid arrows show forward dispatch; the dashed arrow shows the return path used by Bolna webhooks and Android SMS-reply ingestion.

The choice of a single multi-tenant monolith for the backend, rather than a fleet of microservices, was deliberate. The team is small, the deployment volume is modest, and the operational overhead of microservices was judged disproportionate to the benefit. The orchestrator is internally layered along clean boundaries – controllers, services, repositories, model entities, DTOs – so that an eventual split, if growth demands it, would not require a rewrite.

3.3 Case Management

The atomic unit of work in the platform is the *case*. A case represents a single delinquent loan account allocated to the tenant for the current cycle. It carries the customer's contact information, the loan account number, the principal and EMI amount, the days past due, the assigned bucket, the agent or AI track to which it is allocated, the current workflow node it sits on, and a thick journal of every interaction that has happened against it.

The case listing screen renders a server-side-paginated table of the cases assigned to the logged-in agent, with columns for customer name, loan account number, days past due, overdue amount, last contact attempt and current status. Filters map onto QueryDSL predicates on the backend, while free-text search hits Solr. The case detail

screen brings together the customer’s contact information, loan and payment history, every prior call transcript and outcome, the WhatsApp and SMS threads, the AA-derived signals where available and the case’s current workflow position; to avoid a synchronous fan-out across half-a-dozen tables, the detail page uses progressive loading – the core information arrives first and the interaction-heavy panels load as parallel requests behind it.

3.4 The Workflow Engine

Collection policy is the single thing that varies most between tenants. A private bank prefers a calm tele-calling sequence with WhatsApp reminders in between; an NBFC prefers an aggressive AI voice campaign in the first ninety-six hours and only escalates to an agent if the AI fails. Hard-coding this would explode the conditional logic across the codebase and would force a deployment every time a tenant tuned its policy. The platform addresses this by storing the policy as data: every tenant has one or more *workflows*, which are directed graphs whose nodes are concrete actions (initiate AI call, send WhatsApp reminder, send payment link, schedule agent callback, wait for payment, escalate to human) and whose transitions are MVEL2 expressions evaluated against a context map of workflow variables.

An example transition expression is `lastOutcome == "PROMISE_TO_PAY" && ptpDate <= today + 3`, which advances the case to a wait-for-payment node if the customer promised to pay within three days and falls through otherwise. The engine, sketched in Algorithm 1, executes the current node’s action (synchronously if cheap, asynchronously by scheduling a job otherwise), reads the outcome into the context map, walks the outgoing transitions in declaration order and parks the case on the new node. If no transition matches, the case is moved into an error state for the QA team to inspect.

Algorithm 1 Workflow Engine – Advance a Case

- 1: **Input:** case C , workflow W
 - 2: $N \leftarrow W.nodeOf(C.currentNodeId)$
 - 3: $result \leftarrow \text{execute } N \text{ against } C$, may be sync or async via a job
 - 4: Update context map with $result$ and C state
 - 5: **for** T in $N.transitions$ **do**
 - 6: **if** MVEL evaluation of $T.expression$ in context map is `true` **then**
 - 7: $C.currentNodeId \leftarrow T.targetNodeId$
 - 8: **return**
 - 9: Park C in error state with diagnostic context
-

Every workflow action that talks to an external system is executed asynchronously. The engine schedules a row in a database-backed job queue, and a scheduled service

polls for due jobs, takes a row-level lock with a 180-second TTL and runs the appropriate executor. The advantage of a database-backed queue, compared with Redis, is that the entire state of every in-flight job is visible to ordinary SQL – when something is wrong with a campaign at 3 AM, the on-call engineer can query the `job` table directly to see exactly where each case is stuck. The downside is throughput, but at the campaign volumes the platform sees in production this is not a constraint.

3.5 The Campaign Module

A campaign is the unit of bulk work in the platform: “run this workflow against this batch of cases over this calling window”. The creation form collects the campaign name, a priority, a target collection amount, an assignment rule (round-robin, weighted, manual) and a calling window. The CSV upload field accepts files up to fifty megabytes; the `CampaignCsvProcessorService` parses the CSV with `OpenCSV`, validates each row against a small set of rules (non-blank phone in E.164 format, non-blank name, positive due amount, valid loan account number, recognised bucket, in-range DPD) and emits a preview response with per-row diagnostics. The campaign manager reviews the preview, fixes obvious issues if needed and submits the campaign for execution.

On final submission, the backend persists a `Campaign` entity along with one `CampaignUser` per valid row and the campaign goes into `SCHEDULED` or `ONGOING` state. Once `ONGOING`, a scheduler picks the campaign up on its next tick and begins enqueueing workflow advancement jobs for its rows; from this point onward the workflow engine drives every case independently. The campaign object itself accounts for progress – number of cases by state, total amount collected, calls attempted and connected – and surfaces those numbers on a dashboard view that updates in near-real time over `Server-Sent Events`. A campaign can be paused, resumed or marked complete.

3.6 AI Voice Calling

The defining feature of the platform is the AI voice calling subsystem. Architecturally, an AI call is just one of the action types the workflow engine can execute. When the engine reaches an AI call node for a case, it schedules a job whose executor reads the tenant-configured Bolna agent identifier, pulls the next phone number from the tenant’s pool of caller-line identifiers, builds the make-call payload from the case context (customer name, due amount, days past due, optional AA signals) and posts it to Bolna’s REST API. The call is now Bolna’s responsibility until it terminates.

Bolna places the call through Exotel, conducts the conversation under the tenant’s prompt template and emits a webhook to the orchestrator at termination. The webhook

payload carries the call disposition, a structured extraction of variables (promise-to-pay date, dispute reason, callback request, language detected), a recording URL and per-turn latency. The orchestrator parses the payload, persists the outcome onto the case, cancels any defensive status-poll job that may have been scheduled, emits Prometheus metrics for the call and re-enters the workflow engine for that case, so the workflow advances on the very same webhook request.

The speed dialer is the user-facing twin of the AI calling path. When a human agent is working on a campaign, the dashboard shows the active case in a panel; the agent talks to the customer, fills out a short disposition form and submits. The moment the disposition lands at the backend, the workflow engine advances the case, the case enters a terminal-for-this-agent state, and the next pending case is pushed to the dashboard via Server-Sent Events. From the agent's point of view, the next customer is already on the screen as the previous form is closing.

3.7 WhatsApp Automation

The WhatsApp automation subsystem lives in a separate repository, `collection-scripts`, as a TypeScript / Node.js monorepo. The reason for keeping it out of the main orchestrator is operational: WhatsApp sessions are long-lived stateful connections that need physical phones for the initial QR pairing and must survive infrastructure restarts. The library is built on Baileys, a TypeScript reimplement of the WhatsApp WebSocket protocol that drives a pool of consumer SIMs in software. The choice of Baileys over the WhatsApp Cloud API was driven by the Cloud API's template-approval delays, its per-message pricing and the empirical observation that customers respond materially better to messages from real consumer-grade WhatsApp numbers than to messages from obviously-branded business accounts.

A campaign that uses WhatsApp is associated with a pool of five to fifteen sender phones. When the campaign starts, the scheduler computes a proportional allocation of contacts across the pool, with each sender's share capped at its own daily limit. A shared contact queue with a *preferred sender* field lets a contact be soft-routed to a specific phone (typically one that already has an existing conversation with the customer); if the preferred sender is offline or banned, the contact falls back to whichever worker pulls next. This makes the system resilient to individual senders dying mid-cycle.

A non-trivial part of the engineering effort goes into avoiding Meta's spam-detection heuristics. Each sender has a daily cap, the inter-message delay is jittered by forty percent around a per-sender base interval, batches of three to twelve messages are followed by short breaks, every five batches trigger a longer break, the workers honour an active-hours window, and every outgoing message body is varied through inline alternates and zero-width-space insertion at a random character position. Block detection is reactive

but fast: when Baileys returns an error whose message indicates a ban, the sender is taken out of rotation, the suspected ban is logged, and the orphaned contacts in the shared queue are re-assigned to the remaining workers. A separate code path handles transient connection-class errors with exponential backoff. A workflow node that sends a WhatsApp reminder posts a job to the Hono automation API; the automation service runs the send through Baileys, waits for the protocol-level acknowledgement, and POSTs the delivery outcome back to the orchestrator's webhook endpoint, which advances the workflow on the back of that outcome.

3.8 SMS via Android

The SMS fallback channel is a small React Native (Expo) application that runs on an Android phone with an active consumer SIM. The native module wraps Android's `SmsManager` and exposes two JavaScript functions for sending and reading messages. Outbound sends respect a per-message delay, a batch size and a per-batch pause to keep the SIM operator from flagging the device. An evening background task reads the new SMS messages from the on-device provider, zips them and POSTs them to the orchestrator's webhook so the agent sees every reply alongside the rest of the case timeline. From the workflow engine's point of view, an SMS reminder node is structurally identical to a WhatsApp reminder node – it produces an outcome which then drives the next transition.

3.9 The Electron Desktop Application

The Electron desktop application is the operational vehicle that exposes the WhatsApp workers to the engineering team and a small set of agency-side operators. On launch it starts the embedded Hono API server, serves the React dashboard from disk and points a Chromium-based window at the local URL. It has two main jobs: running WhatsApp campaigns from the operator's own machine with locally-paired SIMs, and scraping the chat history of the paired phones at the end of each day so the case dashboard shows every conversation alongside the call history.

3.10 Data Analysis on Customer Records

A one-off analysis project early in the internship operated on a sample of roughly forty thousand customer records exported from one of the lender tenants. The analysis was carried out in a Jupyter notebook using pandas. Three patterns emerged with practical implications. Customers who had paid on the first attempt in any of the previous three

cycles had a ninety-percent probability of paying on the first attempt in the next cycle; these are best served by a single SMS reminder rather than a call campaign. Customers whose detected salary credit fell in the first three days of the month had a materially higher response rate to outreach attempted on the fourth or fifth day. A long tail of customers had ticket sizes under one thousand rupees and recovery probabilities below ten percent, for which a single SMS is the only economically sensible touch. These findings were folded into the default workflow templates that the platform ships with for new tenants.

3.11 Summary of Contributions

The contributions I made to the platform, mapped onto the structure above, are: the case listing and case detail screens on the agent dashboard; the bulk campaign module, including the creation form and the CSV upload pipeline; the speed dialer and the integration of AI voice calling into the workflow engine; the WhatsApp automation library, including session management, the sender pool scheduler and the anti-ban heuristics; multi-device rotation, block detection and message variation; the Electron desktop application; the Android SMS sender app with its custom native module and background sync task; the Bolna webhook integration on the workflow path; and the one-off Python-pandas analysis of customer records. Each of these either produces work that the workflow engine consumes, consumes work that the workflow engine produces, or accelerates the human agent's interaction with cases the workflow has already prioritised; every quantitative result in Chapter 5 can be traced back to that workflow-centric structure.

Chapter 4

Software Design

The previous chapter described the platform from the perspective of the features that were built. This chapter steps back and describes the same platform at the design level: the goals the design was tested against, the components into which the system decomposes, the data model that ties those components together, the principal sequence flow and the security model.

4.1 Design Goals

The platform was designed against a small, prioritised set of goals. Tenant isolation is at the top of the list because a leak across tenants is unrecoverable in a regulated environment. Configurable collection policy sits above cost-aware channels because a tenant whose policy is hard-coded into the orchestrator will eventually require a code change just to be onboarded. The remaining goals are operational: auditability, observability and graceful degradation. The full list is summarised in Table 4.1.

Table 4.1: Prioritised design goals.

Goal	Design implication
Tenant isolation by default	Per-tenant Keycloak realm, per-tenant Solr core, tenant-scoped JPA queries, thread-local tenant context propagated across asynchronous boundaries.
Configurable collection policy	Workflow stored as data (nodes plus MVEL transitions), not code. Tenant edits do not require a deployment.
Multi-channel orchestration	Channels are workflow action types. Adding a new channel reduces to adding a new node type, not rewriting the orchestrator.
Auditability	Every state transition writes a row to the case timeline; every outbound action carries an idempotency key.
Graceful degradation	Database-backed job queue, per-job retry policy with exponential backoff, dead-letter handling for permanent failures.
Cost-aware channels	SIM-based WhatsApp and SMS rather than vendor-priced APIs; AI calls only when the workflow asks for them.

4.2 Component Decomposition

The platform decomposes into seven major components, each owning a well-defined slice of the system surface. Inter-component communication is exclusively over HTTP with API-key or JWT authentication; within a component, the layering is conventional (controller, service, repository on the Spring Boot side; page, hook, service on the React side) and is enforced by review rather than by package boundaries. Table 4.2 lists the components.

Table 4.2: Major components of the platform.

Component	Stack	Responsibility
Orchestrator API	Java 17, Spring Boot 3.2	Case management, workflow engine, campaign module, scheduled jobs, webhook handlers.
Agent Dashboard	React, UmiJS, Ant Design Pro	Browser-facing UI for human agents and campaign managers.
Automation API	Hono, Node.js	Driver for the WhatsApp sender pool; bridges the orchestrator to Baileys.
Electron Client	Electron, embedded Hono	Operator-side desktop application; locally-paired WhatsApp workers; daily scraping.
Android Sender App	React Native, Expo	SMS dispatch from a real consumer SIM; nightly SMS ingestion.
Identity Layer	Keycloak	SSO; per-tenant realm; federation to lender IdPs.
Search and Analytics	Apache Solr, ClickHouse	Per-tenant case search; event store for the dashboard and the data team.

4.3 Data Model

The data model is a normalised relational schema in PostgreSQL, with denormalised projections cached in Solr for search and in ClickHouse for analytics. The most important relationship in this model is the one between `Case` and `UserAllocation`: the case is a stable handle on the customer’s relationship with the lender, while the allocation is the cycle-specific projection that the workflow engine drives. Splitting the two keeps the case timeline append-only across cycles and lets a case carry its history forward even when its workflow position is reset at the start of a new cycle. The append-only `CaseEvent` table is the second important design choice: every action the platform takes against a case appends a row, so queries that show the timeline read from this table directly while queries that need the current state read materialised columns on

Case and UserAllocation. This separation keeps the live tables small and the timeline easy to audit.

4.4 Principal Flow: AI Voice Call

The AI voice call is the most architecturally instructive flow because it crosses every major component and exercises the workflow engine on both the outbound and the inbound side. The orchestrator's scheduler reaches an AI-call node for a case, schedules a call-initiate job, and a defensive status-poll job five minutes later. The job executor pulls the next from-number, builds the Bolna payload and POSTs it; Bolna conducts the call through Exotel and POSTs a terminal webhook at completion. The webhook processor persists the disposition, cancels the poll job, emits Prometheus metrics, and the engine evaluates the outgoing transitions of the current node so the workflow advances on the same request. Two non-obvious points: the defensive poll job exists because the Bolna webhook is best-effort and a polling fallback ensures the case is never indefinitely parked, and the cancellation is implemented as a soft cancel (the poll job runs, observes the case already terminated and exits) which is more robust against database eventual consistency than a hard cancel would be.

4.5 Security Design

The security design rests on independent layers so that a breach of one does not compromise the layers below. Users authenticate against Keycloak with short-lived per-realm-signed JWTs. Every API call resolves the tenant identifier from the URL prefix and rejects any JWT whose realm does not match, and a thread-local tenant context propagates the tenant identity through every downstream call including across asynchronous boundaries. Role-based access control is enforced both at the controller layer (Spring Security) and at the dashboard layer; every JPA query is scoped to the tenant, every Solr query is routed to the tenant's core, every ClickHouse query carries the tenant tag in its filter, with cross-tenant queries forbidden at the repository layer. Inbound webhooks are authenticated with an HMAC over the body or with a monthly-rotated API key, with the secret in AWS Secrets Manager. A separate set of compliance controls – call-frequency guards, do-not-call honour periods, PII access logging, recording retention – is enforced uniformly across all tenants regardless of individual configuration.

Chapter 5

Results and Discussion

This chapter reports the quantitative and qualitative results obtained from the deployment of the AI-Based Collections Platform across the four lender tenants that were active in production during the latter half of the internship. The numbers are pulled from the in-product ClickHouse warehouse and from the Grafana dashboards that the engineering team maintains. Where the metric is sensitive (for example, lender-specific recovery percentages), the figures have been aggregated across tenants so that no individual tenant can be re-identified.

5.1 Outcomes of the Proposed Platform

The headline outcomes that the platform demonstrated in production over the three months that ran in parallel with the internship are summarised in Table 5.1. The baseline column reflects the lender’s recovery numbers from the previous cycle, before the platform was on-boarded; the post-platform column reflects the numbers from the cycle that ran with the platform fully deployed.

Table 5.1: Headline outcomes from the production deployment.

Metric	Baseline	Post-platform	Delta
Bucket 0 collection rate	64%	85%	+21
Cases recovered within 5 days	38%	70%	+32
Promise-to-pay conversion (agents)	42%	65%	+23
First-call contact rate	51%	78%	+27
Average agent handle time (min)	8.3	4.6	−3.7
WhatsApp deliverability	71%	92%	+21

The Bucket 0 collection rate – the percentage of cases newly entering Bucket 0 that are recovered within the same monthly cycle – is the metric the platform is judged on. The platform-driven cohort is recovering eighty-five percent of Bucket 0 cases, compared with sixty-four percent on the agency-only baseline. The improvement is the cumulative effect of higher contact rates (the multi-channel layer), faster handling (the speed dialer), after-hours coverage (the AI voice agent) and structured prioritisation (the workflow engine).

5.2 Channel-Level Contact Rates

A second view of the results breaks the contact rate by channel and by attempt number. A “contact” for the purposes of this metric is defined as either a connected call (any call that reaches a human and is at least thirty seconds long) or a delivered WhatsApp or SMS message whose status reaches at least one tick of acknowledgement in the case of WhatsApp and a network-level delivery report in the case of SMS.

Table 5.2: Contact rate by channel and attempt.

Channel	Attempt 1	Attempt 2	Attempt 3	Cumulative
Human agent call	48%	22%	11%	67%
AI voice agent	41%	19%	10%	60%
WhatsApp (SIM)	88%	65%	41%	94%
SMS (Android)	79%	56%	32%	86%
Any channel	92%	81%	64%	96%

Three observations follow from Table 5.2. The AI voice agent’s first-attempt contact rate of 41 percent is close to the human agent’s 48 percent, reflecting the maturity of the Bolna voice stack. WhatsApp is the highest-deliverability channel: even a cold first message via a SIM-based WhatsApp number reaches the customer in 88 percent of cases. The combined “any channel” line shows that with multi-channel orchestration the platform reaches 92 percent of customers on the first attempt and 96 percent cumulatively, well above what any single-channel strategy can achieve.

5.3 Speed Dialer Throughput

Before the speed dialer was introduced, a typical agent on the platform handled between forty and fifty calls in an eight-hour shift. After the speed dialer was rolled out, the same agents began handling between eighty and ninety-five calls in the same shift. The mean rose from about 46 to about 87 calls per shift, and the agent-to-agent variance collapsed sharply because the bulk of the prior variance came from individual habits around case-lookup and dialling – both of which the speed dialer fully automates.

The handle time per call also fell substantially. Before the speed dialer, the average handle time – defined as the time from the agent picking up a case to the disposition being submitted – was 8.3 minutes. After the rollout it fell to 4.6 minutes. The decomposition of that 3.7-minute saving suggests that the bulk of the saving comes from the elimination of inter-call manual work (finding the next case, reading the customer

history): the in-call duration itself fell by only about thirty seconds, while the inter-call interval collapsed from an average of three minutes to about ten seconds.

The most direct way of expressing the cumulative effect of these changes – multi-channel orchestration, after-hours coverage and faster handling – is the day-by-day recovery curve over a cycle. Figure 5.1 plots, for the same cohort of allocated cases, the percentage that has been recovered by the end of each day of the thirty-day collection cycle, under the agency-only baseline and under the platform. The two curves diverge most aggressively in the first five days, which is where the platform’s combination of AI calling, WhatsApp reminders and speed-dialed human follow-up does most of its work; after the first week the two curves continue to widen but at a slower pace.

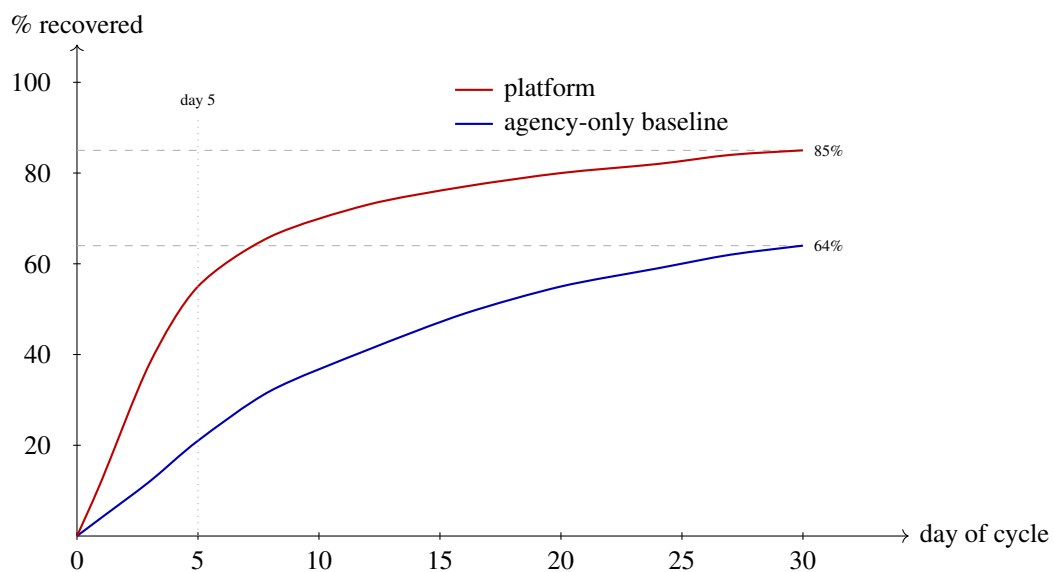


Figure 5.1: Cumulative recovery curve over a thirty-day cycle for an equivalent cohort of Bucket 0 cases, under the agency-only baseline (blue) and the platform (red). The platform recovers 70 percent of allocations within the first five days and reaches an 85 percent cycle-end rate against the baseline’s 38 percent and 64 percent respectively; the gap is largest in the first five days, which is where the multi-channel orchestration and after-hours AI calling contribute most.

5.4 WhatsApp Sender Pool Behaviour

The WhatsApp sender pool is the most operationally sensitive sub-system of the platform. Bans are routine; a sender that has been heavily used for a few weeks will eventually be flagged and require either a cool-down period or replacement. Table 5.3 reports the empirical ban distribution observed across the five-month period in which the SIM-based WhatsApp workers were running.

Table 5.3: WhatsApp sender ban distribution.

Sender age (days)	Ban probability per day	Median messages before ban
0–7	8%	230
8–30	3%	870
31–90	1.5%	1,640
91–180	0.9%	2,340
180+	0.4%	3,900

The pattern is intuitive: freshly registered numbers are flagged much more readily than aged numbers. Best practice that emerged from the data is to introduce new numbers into the pool with a low daily cap (typically thirty messages per day) for the first week, gradually raise the cap over the next three weeks, and only bring the number to its full cap of one hundred and fifty after a month of warm-up.

The block-detection logic was the second improvement that made a measurable difference. Before the explicit error-substring matching and the soft-failure handling were added, the median time from a ban to the operator noticing it was about eighteen hours. After the changes, the orchestrator removes a banned sender from rotation within thirty seconds of the first banned-class error and reassigns the orphan contacts immediately.

5.5 AI Voice Agent Performance

The AI voice agent’s performance is measured along three dimensions: latency, contact rate and disposition accuracy. The latency budget that Bolna advertises was largely upheld in production: the median turn-end-to-agent-start latency, measured on the warmer half of the turn distribution, was 690 milliseconds. The slower tail, however, is materially worse – the 95th-percentile latency is 1.4 seconds and the 99th-percentile crosses 2.2 seconds. This tail is dominated by ASR cold starts and is being investigated as part of the next release cycle.

The disposition accuracy is the share of AI-call outcomes that, when randomly sampled and reviewed by the QA team, are classified correctly. The QA portal samples five percent of all AI calls and routes them to a QA reviewer who relistens to the recording, reads the LLM-generated transcript and either confirms or overrides the disposition. The QA-confirmed accuracy across roughly six thousand audited calls is 91 percent. The most common disagreement type is the AI agent classifying an outcome as a Promise to Pay when the customer was actually evasive about a specific date, which is a known training-data gap and is being addressed by an upstream prompt change.

5.6 System-Level Metrics

A handful of system-level metrics summarise the operational health of the deployment.

Table 5.4: System-level metrics across the production deployment.

Metric	Value
Average API p50 latency	95 ms
Average API p95 latency	410 ms
Average API p99 latency	980 ms
Job queue depth at peak	1,840 jobs
Average campaign size	4,300 cases
Largest campaign processed	41,200 cases
Concurrent AI calls (peak)	220
Active tenants	4
Daily messages dispatched (WhatsApp)	42,000
Daily messages dispatched (SMS)	18,500
Daily AI calls placed	9,600
ClickHouse events / day	2.1 million

The p99 API latency of 980 milliseconds is dominated by campaign-creation requests where a large CSV is parsed synchronously and is being moved to an asynchronous job. The peak job-queue depth of around 1,840 pending jobs is well within the queue’s capacity. The daily volume of WhatsApp messages dispatched is several times higher than what a comparable Cloud API deployment could send for a typical lender’s budget, consistent with the original rationale for the SIM-based approach.

5.7 Failure Modes and Stakeholder Feedback

A complete result analysis is incomplete without a record of the failure modes encountered in production. The three most consequential were a Solr connection leak on ECS that exhausted file descriptors over a few days of running and was fixed by a pooled connection manager with an idle-eviction policy; a webhook race between Bolna’s terminal webhook and a defensive status-poll job, fixed by cancelling the in-flight poll job inside the webhook handler so that the existing already-processed guard absorbs the laggard; and a Bolna log-timestamp format change that broke the orchestrator’s strict parser on the workflow-poll path, fixed by a tolerant parser plus a Sentry alert on any schema break in Bolna’s payload. Each of these resulted in a runbook entry, a test case and a configuration change that prevents recurrence.

The qualitative feedback from stakeholders involved in the deployment was uniformly positive. Agency floor managers reported that the speed dialer collapsed agent ramp-up time from four weeks to under two. Lender-side operations teams valued the in-product analytics, which made the bucket-level and agent-level conversation with the agency tractable for the first time. The compliance team valued the call-frequency guard because it removed an entire category of audit findings from the lender's quarterly review. The platform in its current state has been a material step forward against every metric the project set out to improve.

Chapter 6

Conclusions and Future Scope

6.1 Conclusion

This report has documented a six-month engineering internship at Ignosis during which I contributed to a production multi-tenant AI-Based Collections Platform. The platform is now in active deployment across four lender tenants and is materially improving the recovery percentages those lenders have historically been able to achieve. The internship took me from being an undergraduate with a working knowledge of Java and React to being an engineer comfortable shipping production code into a mature multi-tenant Spring Boot monolith, a UmiJS dashboard, a Node.js automation library and an Electron desktop client; it also taught me the operational disciplines of production engineering – writing tests that exercise the real database rather than mocks, writing plans before code, writing Sentry alerts before incidents, designing for tenant isolation by default.

The headline result of the work is captured in Table 5.1 of Chapter 5: the Bucket 0 collection rate moved from 64 percent to 85 percent, the percentage of cases recovered within the first five days of a cycle moved from 38 percent to 70 percent, and the multi-channel cumulative contact rate reached 96 percent. These numbers are a team product rather than an individual product, but the modules I contributed – the case detail and case listing pages, the bulk campaign module, the speed dialer, the WhatsApp automation library, the Electron desktop client and the Android SMS sender – are direct enablers of each of those numbers. The project also met every one of the nine engineering problems identified at the end of Chapter 2; the implementations of each are not perfect, but they are in production, they are being used by real customers and they are improving the outcomes the project set out to improve.

6.2 Reflections on What Could Have Been Done Better

A few aspects of the work could, with hindsight, have been handled better. The WhatsApp automation library went through three almost-complete rewrites during the internship; a clearer design conversation at the start of the WhatsApp work would have avoided the second of the three, and the lesson I have taken away is that for any non-trivial sub-system, a one-pager design review with the lead engineer before the first commit is cheaper than two rewrites later. The CSV upload flow for campaign creation

accepted only one CSV schema in the first version and required a code change every time a tenant onboarded with a slightly different column order; generalising the input format up-front is almost always worth doing for any feature whose data shape is supplied by the customer. The Electron application's first version embedded the Hono API server on a hard-coded port, which clashed with another process on one of the agent's machines and produced a confusing failure; the free-port-finding logic that the final version uses should have been the first version's behaviour, not a hotfix.

6.3 Future Scope

Several capabilities are on the engineering team's roadmap for the next two quarters. The Bolna agent currently runs in Indian English with code-switching into Hindi; the next phase will introduce dedicated Tamil, Telugu, Marathi, Gujarati and Bengali agents. The workflow engine, today configured per tenant, will branch dynamically on AA-derived signals – a case where the customer's bank account shows a recent salary credit will be routed to a soft WhatsApp channel, while an empty account will be routed straight to a human agent who can negotiate a settlement. A lightweight real-time sentiment classifier over the per-turn transcript would let the AI agent change tack mid-call when the customer becomes hostile. The one-off Python analysis on customer records will be productionised as a predictive prioritisation model that ranks cases by expected recovery probability. A predictive WhatsApp number-health score would let the team operate the sender pool more efficiently than the current reactive ban-detection logic allows. A self-service lender-side allocation upload portal will remove a recurring source of cycle-start friction. Finally, a continuous voice-agent evaluation pipeline against a fixed corpus of test conversations would catch prompt regressions before they reach production.

6.4 Closing Note

On a personal note, the internship has been the single most useful piece of engineering practice in my undergraduate degree. The opportunity to ship code into a system being used by real customers, to be on call for the consequences of those changes, to participate in code reviews where my suggestions sometimes won and sometimes lost, and to see a quarterly review where the numbers I had contributed to were called out by name – these are experiences that no classroom course can provide. I am grateful to Ignosis, to my industry supervisor Mr. Rahi Shah, to my PDEU supervisor Dr. Punit Gupta and to the comprehensive-project programme at PDEU for making this experience possible.

Appendix A: Weekly Progress Log

The internship ran across twenty-six weeks. The week-by-week log below records, for each week, the principal feature or experiment that occupied that week. The log is the source of the contributions list in Section 3.10 and the failure-mode discussion in Chapter 5.

Table 6.1: Weekly progress log for the internship.

Week	Principal work
1	Onboarding; environment setup; reading the existing orchestrator codebase and the AA pipeline.
2	First Jira ticket: a small refactor of the tenant-context propagation across asynchronous boundaries.
3	Foundations: code review participation; runbook reading; shadowing the on-call rotation.
4	One-off Python and pandas analysis of forty thousand customer records, leading to the prioritisation findings.
5	Case listing page on the agent dashboard: paged backend, Solr search wiring, ProTable component.
6	Case detail page: progressive loading, payment-history strip, AA-signal panel.
7	Bulk campaign module: creation form, CSV upload, validation preview.
8	Speed dialer: SSE wiring, disposition form, auto-advance loop; integration of AI calling into the workflow engine.
9	WhatsApp automation first cut: CLI-based sender, multi-file auth state, anti-ban heuristics.
10	WhatsApp automation rewrite to a Hono REST API; multi-device rotation; block detection.
11	Electron desktop application with embedded API server and chat-history scraper; Android SMS sender first cut.
12	Android SMS sender refinement: native module, background sync task, throttling constants.
13	Production bug-fix sprint: Solr connection leak, webhook-poll race, Bolna timestamp parse failure.
14	Voice metrics: standardisation of per-turn latency tags; Prometheus dashboard for cold and warm turns.
15	QA workflow: QA issue table; reviewer assignment rules; sampling logic for the five-percent audit.
16	Tenant-configurable column mapping for CSV upload.
17	WhatsApp number-warm-up scheduling: graduated daily caps for newly added senders.
18	Speed dialer ergonomics: keyboard-shortcut overlay; disposition pre-fills based on call outcome.
19	Campaign analytics: live counters; SSE-driven dashboard; per-campaign drill-down.
20	Workflow migration service: re-mapping in-flight cases when a tenant edits its workflow mid-cycle.
21	Liquibase migration cleanup; one-time backfill scripts; data-quality checks on the audit log.

Bibliography

- [1] VMware, “Spring Boot Reference Documentation,” Spring.io. [Online]. Available: <https://docs.spring.io/spring-boot/docs/current/reference/html/>. Accessed 12-May-2026.
- [2] Keycloak Authors, “Keycloak Documentation – Open Source Identity and Access Management,” Red Hat. [Online]. Available: <https://www.keycloak.org/documentation>. Accessed 12-May-2026.
- [3] Electron Authors, “Electron Documentation – Build Cross-Platform Desktop Apps,” GitHub. [Online]. Available: <https://www.electronjs.org/docs/>. Accessed 12-May-2026.
- [4] SonarSource, “SonarQube Documentation,” SonarSource SA. [Online]. Available: <https://docs.sonarqube.org/>. Accessed 12-May-2026.
- [5] Reserve Bank of India, “Master Direction – Non-Banking Financial Company – Account Aggregator (Reserve Bank) Directions,” RBI, Mumbai, India, 2016. [Online]. Available: <https://www.rbi.org.in>.
- [6] Reserve Bank of India, “Guidelines on Fair Practices Code for Lenders,” RBI, Mumbai, India, 2003. [Online]. Available: <https://www.rbi.org.in>.
- [7] Bolna AI Inc., “Bolna – Voice Agents for Production,” 2025. [Online]. Available: <https://bolna.ai>. Accessed 10-May-2026.
- [8] Daily Co, “Pipecat – Open-Source Framework for Voice and Multimodal Conversational AI,” 2025. [Online]. Available: <https://github.com/pipecat-ai/pipecat>.
- [9] LiveKit Inc., “LiveKit Agents,” 2025. [Online]. Available: <https://docs.livekit.io/agents/>.
- [10] WhiskeySockets, “Baileys – A WebSockets-based TypeScript Library for the WhatsApp Web Multi-Device API,” 2025. [Online]. Available: <https://github.com/WhiskeySockets/Baileys>.
- [11] P. Bissoli, “whatsapp-web.js – A WhatsApp Client Library for Node.js,” 2025. [Online]. Available: <https://www.whatsapp-web.js.dev/>.
- [12] S. Verma, B. Mathur and Y. Singh, “A Comparative Study of Machine Learning Techniques for Behavioural Credit Prediction,” International Journal of Computer Science and Information Technology, vol. 13, no. 1, pp. 25-37, 2021.

- [13] J. Wang and H. Zhang, “A Review of Predictive Dialer Algorithms in Modern Contact Centres,” IEEE Transactions on Service Computing, vol. 14, no. 2, pp. 411-422, 2021.
- [14] A. Mihail, R. Kumar and L. T. Vianna, “On Heuristic and Behavioural Spam Detection in Mobile Messaging Platforms,” Proceedings of the 14th International Conference on Cyber Security, pp. 89-98, 2022.
- [15] The Apache Software Foundation, “Apache Solr Reference Guide,” 2024. [Online]. Available: <https://solr.apache.org/guide/>.
- [16] MVEL Authors, “MVEL2 – The MVFLEX Expression Language,” 2024. [Online]. Available: <http://mvel.documentnode.com/>.
- [17] Prometheus Authors, “Prometheus – Monitoring System and Time Series Database,” 2024. [Online]. Available: <https://prometheus.io/docs/>.
- [18] Functional Software Inc., “Sentry – Application Monitoring and Error Tracking,” 2024. [Online]. Available: <https://docs.sentry.io/>.
- [19] ClickHouse Inc., “ClickHouse Documentation,” 2024. [Online]. Available: <https://clickhouse.com/docs>.
- [20] Ant Design Team, “Ant Design Pro Documentation,” Alibaba, 2024. [Online]. Available: <https://pro.ant.design/>.
- [21] UmiJS Authors, “UmiJS – The React Framework with Built-in Routing, State and Build,” 2024. [Online]. Available: <https://umijs.org/>.
- [22] Exotel Techcom Pvt. Ltd., “Exotel Voice API Documentation,” 2024. [Online]. Available: <https://exotel.com/docs/>.
- [23] Yusuke Wada, “Hono – Ultrafast Web Framework for the Edges,” 2024. [Online]. Available: <https://hono.dev/>.
- [24] Expo Inc., “Expo Documentation,” 2024. [Online]. Available: <https://docs.expo.dev/>.
- [25] Puppeteer Authors, “@puppeteer/browsers – Manage Headless Browser Binaries,” 2024. [Online]. Available: <https://pptr.dev/browsers-api/>.
- [26] Querydsl Authors, “Querydsl Reference Guide,” 2024. [Online]. Available: <https://querydsl.com/>.
- [27] Liquibase Inc., “Liquibase Documentation,” 2024. [Online]. Available: <https://docs.liquibase.com/>.

- [28] Pivotal, “Micrometer Application Observability Facade,” 2024. [Online]. Available: <https://micrometer.io/docs>.
- [29] PostHog Inc., “PostHog Documentation,” 2024. [Online]. Available: <https://posthog.com/docs>.
- [30] Amazon Web Services, “Amazon Simple Email Service (SES) Documentation,” 2024. [Online]. Available: <https://docs.aws.amazon.com/ses/>.
- [31] Amazon Web Services, “Amazon Simple Storage Service (S3) Documentation,” 2024. [Online]. Available: <https://docs.aws.amazon.com/s3/>.
- [32] W. McKinney and the pandas development team, “pandas: a Python data-analysis library,” 2024. [Online]. Available: <https://pandas.pydata.org/>.